

Essay 1b: The Reinforcement Learning Landscape

How do you teach a program to play a game it has never seen before — without telling it what good play looks like? This essay builds up the reinforcement learning framework that makes this possible, from the Bellman equation to policy gradients, tracing a single thread: the credit-assignment problem.

1. The Credit-Assignment Problem

Imagine you just won a game of Ultimate Tic-Tac-Toe after forty moves. You know the outcome — you won — but which of those forty moves deserves the credit? The final move closed out the game, but it was only possible because move thirty-seven created the right global configuration, which itself depended on move twenty-two establishing a threat on the middle local board.

This is the credit-assignment problem, and it is the central challenge in reinforcement learning. An agent takes a sequence of actions, receives a reward at some point (possibly much later), and must figure out which actions caused which rewards. It is the machine-learning equivalent of asking: what, exactly, did you do right?

Everything in this essay — every algorithm, every design choice — is a different answer to that question.

2. Two Dimensions of Value-Based Learning

Before choosing actions, we need a way to evaluate positions. Designing a reinforcement learning algorithm requires making decisions along two dimensions: what exactly we are estimating, and when we update those estimates.

2.1 Dimension 1: What to Estimate (V vs Q)

To evaluate a position, we first need a concept of a policy (denoted by π). You can think of a policy as a mental model or a strategy: it dictates how the agent approaches the game and what moves it is likely to play in any given situation.

With a strategy in mind, we can define the value of a state s . The value, denoted $V_{\pi}(s)$, is the expected total reward the agent will collect if it starts from state s and plays out the rest of the game strictly following its mental model π . Mathematically, it is an expectation (E) over the sum of future rewards:

$$V^\pi(s) = \mathbb{E}_\pi [r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \mid s_t = s]$$

The parameter $\gamma \in (0, 1)$ is the discount factor. Each reward is multiplied by γ raised to the number of steps in the future it occurs. This produces a discounted sum that we can write compactly as:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s \right]$$

Why does γ appear? There are three interlocking reasons, each more compelling than the last.

The first is mathematical convergence. For games that could theoretically go on forever, an infinite sum of rewards would diverge if $\gamma = 1$. By setting $\gamma < 1$, we ensure the math behaves. However, in games like Ultimate Tic-Tac-Toe, the sum actually doesn't go to infinity because the game eventually ends. So why do we still use it?

This brings us to the second, more philosophical reason: uncertainty. Even though the game is finite, a reward ten steps away is contingent on ten more transitions going exactly as expected. Discounting by γ ensures that we focus on securing a reward (like winning the game) earlier rather than later. It is equivalent to assuming the game might suddenly end or slip out of our control at any moment with probability $1 - \gamma$.

The third reason is the one that gives the Bellman equation its power. Consider the optimisation problem we actually want to solve: find the optimal policy π that maximises $\mathbb{E}[\sum \gamma^t r_t]$. This optimisation has a remarkable recursive structure. If you are following the best possible strategy for the whole game, then the moves you make from any point to the end must also be the perfect way to finish. If they weren't, you could just switch to a better finish, which means your original strategy wasn't truly the best one. This is Bellman's principle of optimality, and it implies that V satisfies:

$$V^*(s) = \max_a [r(s, a) + \gamma \cdot \mathbb{E}[V^*(s')]]$$

This is the Bellman optimality equation. It says: the value of the best action from s equals the immediate reward plus the discounted value of wherever that action takes you.

Here, s' represents the **next state** — the state the environment transitions into after taking action a from state s . Because the environment or the opponent might be unpredictable, s' is technically a random variable, which is why we take the expectation E over it.

This equation gives us a very concrete way to play the game. If we somehow magically knew the perfect value function V , *we wouldn't need to do any deep thinking or search during the game. At any state s , we would just simulate every legal action a , look at the resulting next states s' , and pick the action that gives us the highest expected score. A perfect V acts as a perfect compass.*

So the entire goal of reinforcement learning boils down to one task: **solving the Bellman Equation** to find V^* . Because the state space of a game like Ultimate Tic-Tac-Toe is far too large to solve this equation exactly, the rest of this essay explores algorithms like Temporal Difference (TD) learning. These algorithms are essentially iterative methods that try to bump our current, imperfect estimates of V until they satisfy this very equation.

What's the mathematical justification for this recursive form? It falls out directly from the law of iterated expectations — what probabilists call the tower property. If $G_t = r_t + \gamma \cdot r_{t+1} + \gamma^2 \cdot r_{t+2} + \dots$ denotes the return from time t , then:

$$G_t = r_t + \gamma \cdot G_{t+1}$$

Taking expectations:

$$V(s_t) = \mathbb{E}[r_t + \gamma G_{t+1} \mid s_t] = \mathbb{E}[r_t \mid s_t] + \gamma \mathbb{E}[\mathbb{E}[G_{t+1} \mid s_{t+1}] \mid s_t] = \mathbb{E}[r_t \mid s_t] + \gamma \mathbb{E}[V(s_{t+1}) \mid s_t]$$

The inner expectation $\mathbb{E}[G_{t+1} \mid s_{t+1}]$ is just $V(s_{t+1})$ by definition; the tower property lets us collapse it. The Bellman equation is, in this sense, nothing more than the law of iterated expectations applied to discounted sums.

From States to Actions: Introducing Q

While $V(s)$ tells us how good it is to be in a state s , it has a practical limitation. As we saw earlier, if you only know V , finding the best next move requires simulating every possible action to see what the next state s' will be, and then looking up $V(s')$. This requires you to perfectly know the rules of the game—what we call having a *model* of

the environment.

To avoid this, we can define a slightly different value function: $Q(s, a)$. The Q-function tells us the expected total reward if we start in state s , take a specific action a , and *then* follow our policy for the rest of the game.

Because $Q(s, a)$ already has the action baked into it, making a decision becomes trivial: you just look at your current state s , check the Q-values for all available actions a , and pick the action with the highest number. You don't need to simulate the future or know the rules of the game at all; the Q-function does the heavy lifting for you.

2.2 Dimension 2: When to Update (Monte Carlo vs TD)

This determines the entire character of the learning algorithm. You are in state s_t . You have taken some actions. At some point you must use your observations to update your value estimates. The question is: how much of the future do you observe before making that update?

One extreme is to play the entire game to completion, collect the final outcome, and then work backward attributing credit. This is the Monte Carlo approach.

The other extreme is to take a single step, observe the immediate reward and the new state, and update immediately. This is the Temporal Difference approach.

Both are legitimate strategies with genuinely different statistical properties.

Monte Carlo: Learn from Complete Games The simplest strategy for the credit-assignment problem is to stop worrying about it during the game, play all the way to the end, and only then assign credit.

Define the return from time t as the discounted sum of all future rewards:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^{T-t} r_T$$

where T is the final timestep. After the game ends, we know G_t exactly for every timestep t . We then update our value estimate at each state visited:

$$V(s_t) \leftarrow V(s_t) + \alpha [G_t - V(s_t)]$$

This is an exponentially weighted running average: each update moves $V(s_t)$ a fraction α toward the observed return.

Monte Carlo estimation is unbiased: because G_t is the actual return from the actual game, not an estimate of an estimate, we are moving toward the true value on average. The cost is variance: the return G_t depends on every subsequent action and every subsequent environment transition. In a forty-move game, G_t is a function of thirty-nine more steps, each of which could have gone differently. This makes individual estimates noisy, and convergence slow.

The deeper problem is credit assignment. Every state in the game gets updated, but only the terminal state receives a nonzero reward directly. The update for the first move is driven entirely by the cumulative return G_0 — a noisy signal that reflects the quality of thirty-nine more decisions. The first move is not well-served by this.

TD Learning: Learn from One Step Temporal Difference learning makes a different bet. Instead of waiting for the game to end, it bootstraps: it uses its current estimate of future value to construct a training target.

After observing a transition $s_t \rightarrow a_t \rightarrow r_t \rightarrow s_{t+1}$, the TD update is:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

This quantity δ_t is the TD error — the gap between what we expected ($V(s_t)$) and what we observed plus what we now expect from the next state ($r_t + \gamma \cdot V(s_{t+1})$). We update:

$$V(s_t) \leftarrow V(s_t) + \alpha \delta_t$$

TD learning can update after every single step, not just at the end of a game. Its update target is biased — it uses $V(s_{t+1})$, which is itself an estimate, not the true value — but it is much lower variance than Monte Carlo, because the target only looks one step into the future.

There is a subtlety here worth pausing on. At the start of training, we initialise $V(s) = 0$ for every state. Consider a game where all rewards are zero except upon entering the terminal state, where the winner receives +1. Now follow the TD update rule:

On the first game, every transition has $r_t = 0$ and $V(s_{t+1}) = 0$. So $\delta_t = 0 + \gamma \cdot 0 - 0 = 0$. The update is zero. Nothing is learned from any non-terminal state.

For the final winning move from state s_{t-1} into the terminal state s_t , the reward is $r_t = 1$. Since s_t is terminal, it has no future value: $V(s_t) = 0$. The TD error for the state

just before the end is $\delta_{t-1} = 1 + 0 - 0 = 1$. Thus, the state just prior to winning gets updated: $V(s_{t-1})$ moves from 0 to something positive.

On the second game, if the agent happens to visit the same s_{t-1} again, it will find $V(s_{t-1}) > 0$, and the state just before that will now have a nonzero target. The signal is propagating backward — but only one step per game.

In theory, over many games, this “correction wave” propagates from terminal states backward through the game tree, eventually giving every state an accurate value estimate. In practice, for Ultimate Tic-Tac-Toe, the wave never arrives.

The problem is the 3^{81} state space. There are roughly 4.4×10^{38} possible board configurations. Every game traces a path through this enormous space, and — crucially — almost every game traces a different path. The probability that any two games share a non-terminal state is vanishingly small. The correction learned from one game has nowhere to propagate to.

This is not a problem that more games will fix. You could play a billion games and still have visited a negligible fraction of the 3^{81} possible states. The tabular approach — storing one number per state — is simply not viable for a game of this scale.

What is needed is a way to generalise: to update the value of one state and have that update automatically propagate to similar states. This is what neural networks provide. A neural network parameterises $V(s)$ as a function of the state — shared weights mean that an update for one state influences the estimated values of all similar states. In a grid game like Ultimate Tic-Tac-Toe, Convolutional Neural Networks (CNNs) are especially powerful for this: a win threat in the top-left corner is structurally identical to a win threat in the bottom-right corner, and convolutions allow the network to share that learned knowledge across the entire board. Similarly, Transformer architectures are an excellent fit here: they unlock long-distance attention mechanisms, which is perfect for understanding how a move in one sub-board affects the global game state across distant sub-boards. In either case, the correction wave becomes, in effect, a gradient that flows through the entire function class at once. We will return to this in later essays. For now, the lesson is: tabular TD learning is theoretically clean but practically unusable at scale.

Q-Learning Rather than estimating $V(s)$, it is often more useful to estimate $Q(s, a)$ — the value of taking action a from state s , then playing optimally thereafter.

Q-learning updates Q directly:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

The target $r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a')$ is the “greedy” TD target: it uses the best Q-value available at the next state, regardless of what action the agent actually took. This makes Q-learning an off-policy algorithm — it learns the optimal policy even while following a different (e.g. exploratory) one.

Q-learning enjoys strong convergence guarantees in the tabular case. But it inherits exactly the same scaling problem as tabular V-learning: storing one number per (state, action) pair is impossible when the state space is 3^{81} .

Deep Q-Networks (DQN) The fix is to replace the Q-table with a neural network $Q(s, a; \theta)$, parameterised by weights θ . Given a state s (encoded as a vector or tensor), the network outputs Q-values for every action simultaneously.

Training proceeds by minimising the loss:

$$L(\theta) = \mathbb{E} \left[(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2 \right]$$

where θ^- are the weights of a target network — a periodically frozen copy of the main network. The target network stabilises training: without it, the target itself changes every gradient step, leading to oscillation or divergence.

DQN also uses an experience replay buffer: past transitions (s, a, r, s') are stored and sampled randomly for training. Random sampling breaks the temporal correlations in sequential game data, which would otherwise bias the gradient estimates.

This combination — neural function approximation + target network + experience replay — is what made deep reinforcement learning practical. DeepMind’s DQN paper (2015) demonstrated superhuman performance on dozens of Atari games using only raw pixels as input.

For UTTT, DQN is a natural starting point. But it has a limitation: it is purely value-based. It estimates how good each action is, and acts greedily. It says nothing about how to choose actions when many are similarly valued, or how to reason about the structure of the action space.

3. Policy Gradients and REINFORCE

An alternative approach is to directly parameterise the policy — the function that maps states to actions. Instead of learning V or Q and deriving a policy from them, we learn $\pi_\theta(a|s)$ directly: a probability distribution over actions given the current state.

The objective is simple to state: maximise expected return.

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_0] = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right]$$

$J(\theta)$ is the expected discounted total reward when the agent follows policy π_θ from the start. We want to find $\theta = \operatorname{argmax}_\theta J(\theta)$.

To maximise $J(\theta)$, we need its gradient with respect to θ . The policy gradient theorem gives:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) G_t \right]$$

The key object is $\nabla_\theta \log \pi_\theta(a_t | s_t)$ — the gradient of the log-probability of the action taken, with respect to the policy parameters. This is called the score function. The intuition: if G_t is large (the game went well), we want to increase the log-probability of the actions we took. If G_t is small, we want to decrease them. The score function tells us which direction to push θ .

Why $\log \pi$ rather than π ? Because $\nabla_\theta \log \pi_\theta = \nabla_\theta \pi_\theta / \pi_\theta$. When we take an expectation over actions (which are sampled from π_θ), the π_θ in the denominator cancels the π_θ in the sampling measure, giving a clean gradient estimate that we can compute from samples.

REINFORCE is the algorithm that implements this directly: play a complete game, compute G_t for every step, and update θ by gradient ascent. It is Monte Carlo policy gradient — it requires the full return G_t , not an estimate.

REINFORCE is unbiased: since G_t is the actual return from the actual game, the gradient estimate is correct on average. The cost, again, is variance. G_t is influenced by everything that happens after time t , including many things unrelated to the quality of action a_t . The signal is noisy, convergence is slow, and large numbers of games are required.

4. Actor-Critic, GAE, and PPO

The actor-critic architecture is to policy gradients what TD is to Monte Carlo: it replaces the full return G_t with a bootstrapped estimate, reducing variance at the cost of some bias.

The actor is the policy $\pi_\theta(a|s)$. The critic is a value function $V_\phi(s)$, parameterised separately, whose job is to estimate how good each state is. (As a sneak peek: the AlphaZero network we will build later in this series is an actor-critic architecture! It uses a single shared neural network body that splits into two “heads” — one predicting the value, and the other predicting the move probabilities.)

Instead of weighting the score function by G_t , actor-critic weights it by the advantage:

$$\hat{A}_t = G_t - V_\phi(s_t)$$

The advantage says: was this action better or worse than what we expected from this state? Subtracting $V_\phi(s_t)$ does not change the expected gradient ($V_\phi(s_t)$ is independent of the actions taken from s_t), but it dramatically reduces variance by removing the component of G_t that is due to the quality of the state rather than the quality of the action.

In the simplest one-step actor-critic, the advantage is estimated by the TD error:

$$\hat{A}_t \approx \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

This mirrors exactly the MC/TD trade-off we saw for value functions: REINFORCE is MC policy gradient, actor-critic is TD policy gradient.

Generalised Advantage Estimation (GAE)

Between the one-step TD estimate (low variance, high bias) and the full Monte Carlo return (high variance, low bias) is a continuum parameterised by $\lambda \in [0, 1]$:

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

where $\delta_{t+l} = r_{t+l} + \gamma \cdot V(s_{t+l+1}) - V(s_{t+l})$ is the TD error at step $t+l$.

When $\lambda = 0$, this collapses to the one-step TD error δ_t . When $\lambda = 1$, it becomes the full Monte Carlo return minus the baseline. Any $\lambda \in (0, 1)$ interpolates smoothly between

the two. GAE is now standard in high-performance RL systems.

Proximal Policy Optimisation (PPO)

A practical problem with policy gradient methods is that gradient steps can be too large, accidentally destroying a good policy. PPO addresses this by constraining how much the policy can change in a single update.

Define the probability ratio:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$$

If $r_t > 1$, the new policy is more likely to take action a_t than the old one was. PPO's clipped objective is:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

The min of the unclipped and clipped versions ensures that the policy is only pushed in a beneficial direction, and never too far. If an action was advantageous ($\hat{A}_t > 0$), the objective rewards increasing its probability — but only up to a factor of $1 + \epsilon$. If an action was disadvantageous, the objective penalises increasing its probability — but cannot benefit from decreasing it past $1 - \epsilon$.

PPO is the most widely used policy gradient algorithm in practice. It is the backbone of RLHF.

RLHF: Reinforcement Learning from Human Feedback

A brief note on the connection to large language models. Reinforcement Learning from Human Feedback (RLHF) is the technique that took GPT-3 to ChatGPT and is responsible for much of the “helpful, harmless, honest” behaviour of modern AI assistants.

The pipeline works in three stages. First, a language model is pretrained on text in the usual way. Second, human raters compare pairs of model outputs and express preferences (“response A is better than response B”). These preferences are used to train a reward model — a separate neural network that predicts which responses humans will prefer. Third, the language model is fine-tuned with PPO, using the reward model as the reward signal. The policy (the language model) is optimised to generate outputs that score highly according to the reward model — which, by construction,

reflects human preferences.

The actor is the language model. The critic is the reward model. The advantage is estimated from the critic's scores. The clipping in PPO prevents the fine-tuned model from drifting too far from the pretrained model, preserving its broad language abilities while nudging it toward human-preferred behaviour.

While PPO has become the standard for training language models to chat, it is still fundamentally a reactive algorithm. To master deep tactical games like Chess, Go, or Ultimate Tic-Tac-Toe, we take a different path. We need an actor-critic algorithm that doesn't just react, but deliberates and plans ahead.

5. The Remaining Problem

We now have a rich toolkit: TD learning, Q-learning, DQN, REINFORCE, actor-critic, GAE, PPO. All of them are principled solutions to the credit-assignment problem. All of them have been demonstrated to work, in some domain, at scale.

None of them, applied naively to Ultimate Tic-Tac-Toe, produces a strong player.

The issue is that all of these methods are fundamentally reactive: they learn to evaluate or act upon states they have encountered. They do not, by themselves, engage in deliberate search. A DQN agent chooses the action with the highest Q-value; it does not look ahead three moves and ask whether that action will create an unavoidable threat. A policy gradient agent samples an action from $\pi_\theta(a|s)$; it does not simulate the opponent's best response.

In games with deep tactical structure — games where the right move depends on reading opponent responses several turns ahead — this is a serious limitation. What we need is a way to combine learned value estimates with explicit forward search.

That combination is Monte Carlo Tree Search — and it is the subject of the next essay.

Next: Essay 1c introduces the multi-armed bandit problem and shows how the explore/exploit trade-off that governs slot machines also governs tree search. The UCB formula that solves the bandit problem turns out to be the key ingredient that makes MCTS both principled and practical.

Code: [Notebook 1 — The UTTT Game Engine](#) shows the full 7-channel `encode_state()`

function in action, including a live demonstration of each channel's content for a real game position.